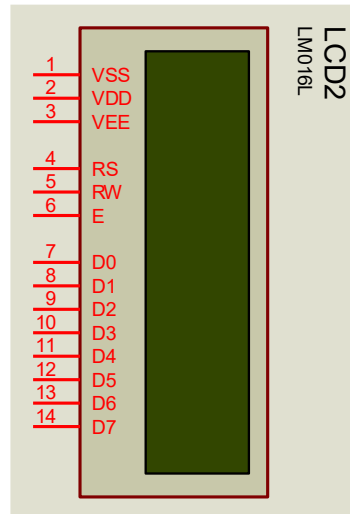


LCD1602

1、管脚定义



VSS:电源地

VDD: 电源正极

VEE: 调节显示亮度，接滑动变电阻

RS: 数据/命令选择端（H/L）

RW: 读写选择端（H/L）

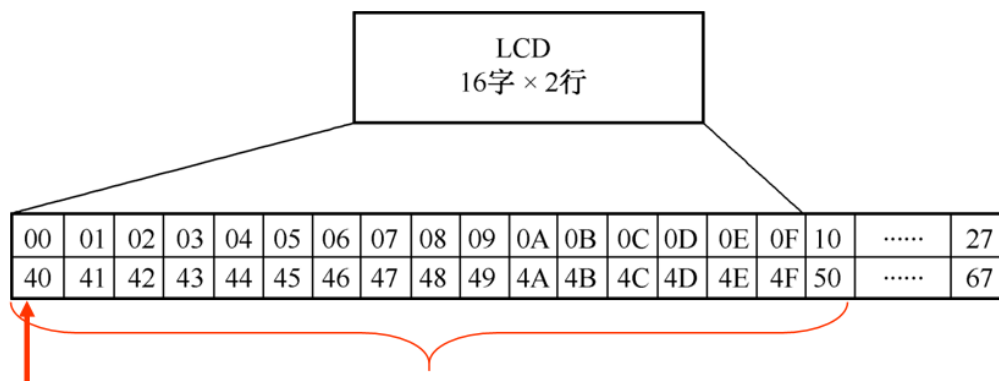
E:使能信号，高电平有效

D0~D7: 数据口

2、基本操作功能

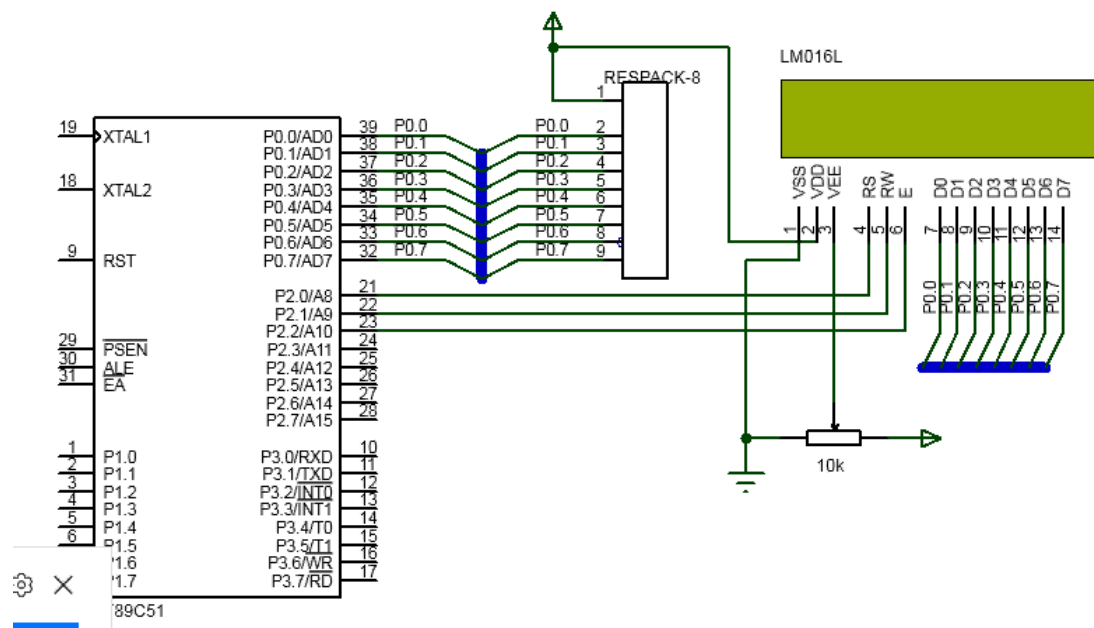
RW(读/写)	RS(数据/命令)	E	功能
0	0	↑	写命令
0	1	↑	写数据
1	0	1	读状态
1	1	1	读数据

3、LCD1602 的地址映射图



每行的起始地址

可以向任何一个地方写入待显示数据，但是在 10H 和 50H 以后需要通过移动屏幕才能显示出来。



4、LCD 状态字说明

位序号	D7	D6	D5	D4	D3	D2	D1	D0
位功能	读写操作使能	当前地址指针的数值						

1: 禁止, 0: 允许

5、初始化

① 显示模式设置

指令码: **38H**

功能: 设置 16*2 显示, 5*7 点阵, 8 位数据接口

② 显示开/关及光标设置

显示开关: **0FH**

写字符后地址加 1: 06H

光标左移: 10H

光标右移: 14H

整屏左移, 同时光标跟随移动: 18H

整屏右移, 同时光标跟随移动: 1CH

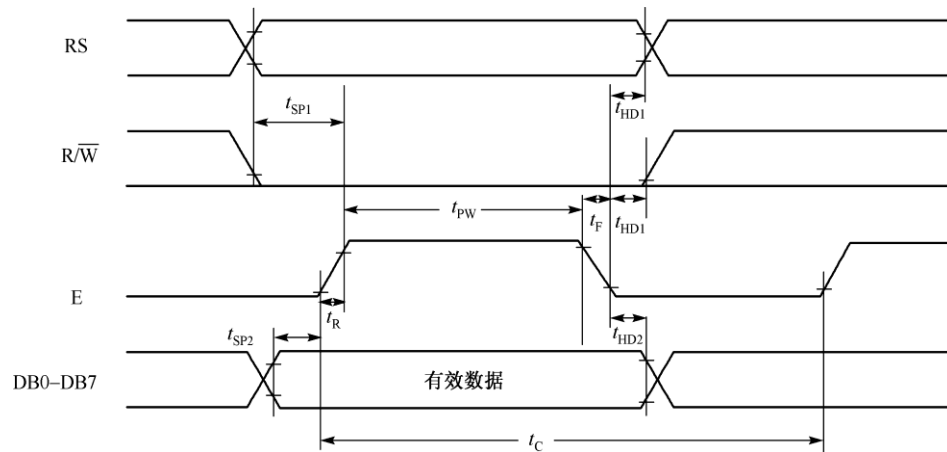
③指针初始化

设置数据地址指针的指令码:

80H+地址码 (0~27H, 40~67H)

6、读写操作和函数代码

LCD1602 的写操作时序图



1 基本操作时序：

- | | |
|---|--------------|
| 1.1 读状态：输入：RS=L, RW=H, E=H | 输出：D0~D7=状态字 |
| 1.2 写指令：输入：RS=L, RW=L, D0~D7=指令码, E=高脉冲 | 输出：无 |
| 1.3 读数据：输入：RS=H, RW=H, E=H | 输出：D0~D7=数据 |
| 1.4 写数据：输入：RS=H, RW=L, D0~D7=数据, E=高脉冲 | 输出：无 |

写操作

- * 写指令的时候，RS 置为低电平，RW 置为低电平，E 置为低电平，然后将指令数据送到数据口 D0~D7，延时 t_{SP1} ，让 1602 准备接收数据，这时候将 E 拉高，产生一个上升沿，这时候指令就开始写入 LCD，延时一段时间，将 E 置低电平。
- * 写数据的时候，RS 置为高电平，RW 置为低电平，E 置为低电平，然后将指令数据送到数据口 D0~D7，延时 t_{SP1} ，让 1602 准备接收数据，这时候将 E 拉高，产生一个上升沿，这时候数据就开始写入 LCD，延时一段时间，将 E 置低电平。
- * 写指令代码

```
* 1. void LCD_WriteCommand(unsigned char Command)
* 2. {
* 3.     LCD_RS = 0; //指令
* 4.     LCD_RW = 0; //写
* 5.     LCD_DataPort = Command; //数据存放
* 6.     LCD_E = 1; //使能
* 7.     Delay1ms(); //最低要求延迟 150 纳秒 我们直接给 1ms
* 8.     LCD_E = 0; //失能
* 9.     Delay1ms();
* 10. }
```

* 写数据代码

```
* 1. void LCD_WriteData(unsigned char Data)
* 2. {
* 3.     LCD_RS = 1; //数据
* 4.     LCD_RW = 0; //写
* 5.     LCD_DataPort = Data; //数据存放
* 6.     LCD_E = 1; //使能
* 7.     Delay1ms(); //最低要求延迟 150 纳秒 我们直接给 1ms
* 8.     LCD_E = 0; //失能
* 9.     Delay1ms();
* 10. }
```

读操作

* 读状态/忙信号的时候：**RS 置为低电平**，**RW 置为高电平**，E 置为低电平。首先将数据口 D0~D7 置为高电平（0xFF，准备读取），然后将 E 拉高，产生一个高电平，LCD 会将状态数据送出到数据口。读取数据口的状态（最高位 D7 为 1 表示忙，0 表示空闲），延时一段数据建立时间，最后将 E 置低电平。

✱ 读数据的时候： **RS 置为高电平**，**RW 置为高电平**，**E 置为低电平**。首先将数据口 D0~D7 置为高电平（0xFF，准备读取），然后将 EN 拉高，LCD 会将 RAM 中的数据送出到数据口。读取数据口的数据，延时一段数据建立时间，最后将 E 置低电平。

读状态/判忙代码

```
1. unsigned char LCD_ReadBusy()
2. {
3.     unsigned char Status;
4.
5.     LCD_DataPort = 0xFF; // 读操作前，先将端口置 1（针对 51 单片机准双向口释放总线）
6.     LCD_RS = 0; // 指令/状态
7.     LCD_RW = 1; // 读
8.     LCD_E = 1; // 使能
9.     Delay1ms(); // 等待数据稳定
10.    Status = LCD_DataPort; // 读取端口数据（包含忙标志 BF 和地址计数器 AC）
11.    LCD_E = 0; // 失能
12.    Delay1ms();
13.
14.    return Status; // 返回状态值，通常判断（Status & 0x80）是否为 1
15. }
16.
```

读数据代码

```
1. unsigned char LCD_ReadData()
2. {
3.     unsigned char Data;
4.
5.     LCD_DataPort = 0xFF; // 读操作前，先将端口置 1（释放总线）
6.     LCD_RS = 1; // 数据
7.     LCD_RW = 1; // 读
8.     LCD_E = 1; // 使能
9.     Delay1ms(); // 等待数据稳定
10.    Data = LCD_DataPort; // 读取 RAM 中的数据
11.    LCD_E = 0; // 失能
12.    Delay1ms();
13.}
```

```

14.     return Data; // 返回读取到的数据
15. }
16.

```

7、显示和定位

* 设置从 (x,y) 开始显示

	显示位置	1	2	3	4	5	6	7		40
DDRAM	第一行	00H	01H	02H	03H	04H	05H	06H		27H
地 址	第二行	40H	41H	42H	43H	44H	45H	46H		67H

LCD 规定：凡是“设定光标地址”的指令，**最高位（D7）必须是 1。**

我们常用的地址（比如 00H 或 40H）最高位都是 0。

所以，为了让 LCD 知道“这是一个定位指令”，我们必须人为地给地址加上 **80H**（二进制 1000 0000）。

公式：

$$\text{实际发送的指令} = \text{目标地址} + 0x80$$

参考代码

```

1. // Line: 行号 (1 或 2)
2. // Column: 列号 (1 ~ 16)
3. void LCD_SetCursor(unsigned char Line, unsigned char Column)
4. {
5.     unsigned char Address;
6.
7.     // 1. 先根据行号确定基准地址 (查 PPT 的表)
8.     if(Line == 1)
9.     {
10.         Address = 0x00; // 第一行起始地址
11.     }

```



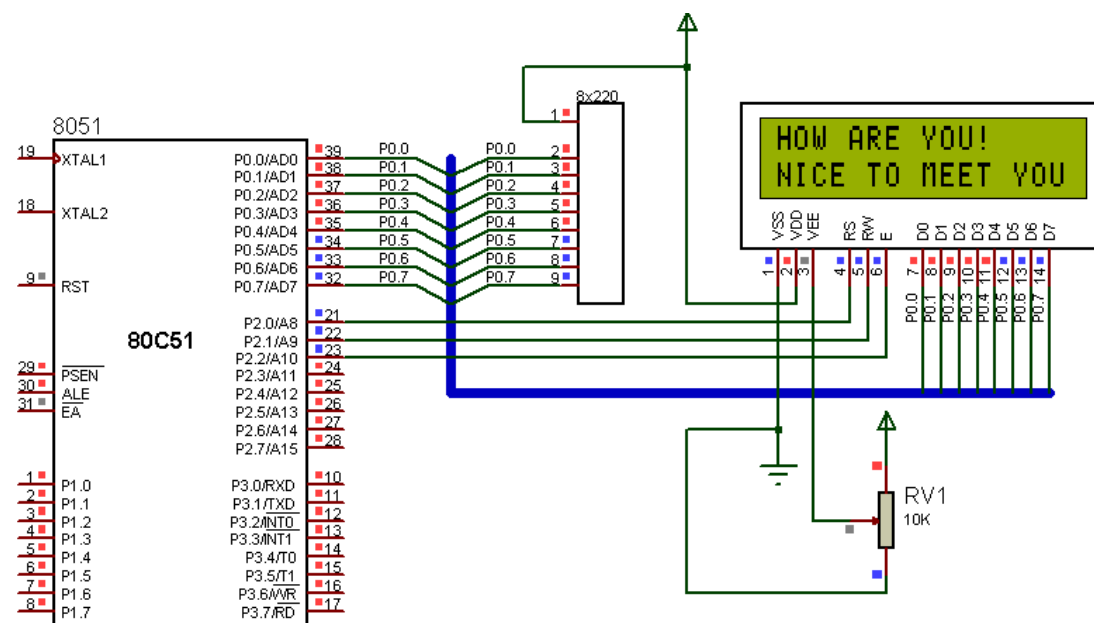
```

12.     else
13.     {
14.         Address = 0x40; // 第二行起始地址
15.     }
16.
17.     // 2. 加上列的偏移量 (列号减 1, 因为地址是从 0 开始算的)
18.     Address = Address + (Column - 1);
19.
20.     // 3. 关键一步: 加上 0x80, 变成“设置地址指令” (PPT 的核心重点)
21.     LCD_WriteCommand(Address + 0x80);
22. }

```

8、例题

利用 LCD1602 显示两行字符。要求光标闪烁，从右移入



配置

```

1. #include<reg51.h>
2. sbit lcden=P2^2;
3. sbit write=P2^1;
4. sbit lcdrs=P2^0;
5. unsigned char code table[]="HOW ARE YOU!";
6. unsigned char code table1[]="NICE TO MEET YOU";
7. unsigned char num;
8. void delays (unsigned int z)
9. { unsigned int x,y;
10.   for (x=z;x>0;x--)
11.     for (y=115;y>0;y--);

```

```
12. }  
13.
```

写指令

```
1. void write_com(unsigned char com)  
2. { lcden=0;  
3. lcdrs=0;           //写命令  
4. write=0;  
5. P0=com;           //送命令  
6. delayms(5);  
7. lcden=1;           //产生正脉冲，使能  
8. delayms(5);  
9. lcden=0;           //拉低  
10. }  
11.
```

写数据

```
1. void write_data (unsigned char dat)  
2. { lcden=0;  
3. lcdrs=1;  
4. write=0;  
5. P0=dat;  
6. delayms(5);  
7. lcden=1;  
8. delayms(5);  
9. lcden=0;  
10. }  
11.
```

初始化 LCD1602

```
1. void init1602()  
2. {  
3.     lcden=0;  
4.     write_com(0x38);           //显示 16*2, 5*7 点阵  
5.     write_com(0x0f);           //开显示，显示光标，光标闪烁  
6.     write_com(0x06);           //写字符后地址加 1  
7.     write_com(0x01);           //显示清屏
```

主函数，开始显示

```
1. void main()
2. {
3.     write=0;
4.     init1602();                //初始化 1602 液晶
5.     write_com(0x80+0x10);      //初始化指针在 10H 处
6.     for(num=0; num<12; num++)
7.     {
8.         write_data(table[num]); //HOW ARE YOU!
9.         delayms(50);
10.    }
11.    write_com(0x80+0x50);
12.    for(num=0; num<16; num++)
13.    {
14.        write_data(table1[num]); //NICE TO MEET YOU.
15.        delayms(50);
16.    }
17.    for (num=0; num<16; num++)
18.    {
19.        write_com(0x18);        //整屏左移，且光标移动
20.        delayms(500);
21.    }
22.    while(1);
23. }
```